Drone Inspection and Navigation in Cluttered Environments

John Hoang, Diego Augustincic, and Eric Zhu

***Abstract*. Reliable autonomous drone flight indoors without GPS or external tracking remains an open problem due to cluttered environments, costly components and tight manoeuvring requirements. This paper presents a low-cost autonomous drone that builds a map of its environment, plans a route through it and flies that route without operator input. A single stereo camera is its only sensor for perceiving its surroundings and localising its position. Perception and planning run on a Raspberry Pi 5, which sends position setpoints to a Pixhawk 4 flight controller that handles the flight dynamics. The drone is enclosed in a lightweight carbon fibre cage for bump and crash tolerance and its parts cost about 1115 Australian dollars. Against motion-capture ground truth, the onboard position estimate stayed within 2.9 cm RMS horizontally, although its height drifted by about 15 cm per minute. In autonomous flights the drone passed through a gap leaving 1.5–5 cm on each side of its cage, crossed a doorway and climbed 2.1 m up a stairwell. One doorway approach failed when stereo depth errors on a textureless wall closed the opening in the map. The drone recovered from pushes on its cage of up to about 200 N, and hover endurance was about seven minutes. The results show that stereo-only autonomy on low-cost hardware is feasible in cluttered indoor spaces, and that map reliability at narrow openings, height drift and endurance are the remaining limits.**

I. Introduction

THE proliferation of unmanned aerial vehicles is transforming industries ranging from commercial delivery to emergency search and rescue, infrastructure inspection and surveillance. While outdoor drone navigation has matured considerably, indoor drone navigation continues to be brittle due to the lack of GPS, confined spaces and the obstacles present in indoor environments.

Indoor autonomous quadcopters offer significant advantages over humans when inspecting confined spaces such as boilers, storage tanks, silos, ducts and pressure vessels, since drones can be built to be compact and manoeuvrable. They can also reach dangerously high places, such as large warehouse or factory roofs, to inspect for ponding water, membrane damage, failed seams and blocked drains. This means fewer people working at height or in confined spaces, faster inspections, less downtime, and repeatable imagery that can be compared over time.

This project presents a drone platform for autonomous interior infrastructure inspection that navigates through cluttered and confined spaces to reach places a human would have difficulty travelling through. The project proposal set four aims, each with a measurable target:

1) *Precise GPS-free localisation:* horizontal error small enough for a 0.5 m drone to pass a 0.6 m gap, i.e. within about 5 cm.
2) *Autonomous navigation through cluttered environments:* reach an operator-selected goal through 0.6 m gaps, doorways and changes in height.
3) *Bump tolerant design:* a cage within 0.5 m tip to tip that protects the drone when it contacts obstacles.
4) *Low cost design:* a complete platform under AUD 1000 that supports its payload for a 10 minute flight.

Final Year Project B paper, submitted October 2026.

The research question is therefore whether a low-cost drone, budgeted at AUD 1000, using a single stereo camera as its only exteroceptive sensor and a Raspberry Pi as its only onboard computer, can localise, map and navigate autonomously through cluttered indoor spaces, including openings only slightly wider than itself. The contributions of this paper are: (i) a complete open-source-based stereo-only navigation system, flown through gaps with as little as 1.5 cm on each side and up a stairwell; (ii) a flight architecture that keeps SLAM corrections out of the control loop; (iii) a numerical and experimental evaluation of a low-cost friction-fit carbon fibre cage; and (iv) an honest account of where the approach fails. Sections II–VI cover related work, system design, test methods, results and conclusions.

II. Background and Related Work

A. Sensing and Obstacle Detection

Without GPS, a UAV must estimate its position using only its onboard perception and processing. The two most common sensing approaches are stereo cameras and LiDAR, both limited by their update rate, processing cost and the vehicle's motion and vibration. LiDAR provides accurate ranging and integrates well into SLAM, but it can be expensive and too large for small platforms. Stereo cameras perceive both depth and visual features while remaining lightweight and low cost, but they rely heavily on sufficient lighting and texture [1], [2]. Schmid et al. showed that stereo vision alone can support indoor and outdoor navigation of a flying robot, using dedicated onboard stereo processing hardware [3].

A stereo camera's limited field of view (FOV) and need for distinct features mean obstacles beside, above, below or behind the UAV may remain unseen until they are too close, and adding cameras carries significant financial, computational and power cost. Basnet et al. instead extended coverage in confined spaces with VL53L8CX infrared Time-of-Flight (ToF) sensors, whose 8×8 ranging grid provides limited detail but inexpensive close-range detection [4]. ToF sensors can give misleading, degraded or absent readings on transparent and reflective materials. Ultrasonic sensing does not share these optical limitations but is susceptible to motor vibration; MEMS-based systems such as BatDeck have demonstrated low-power obstacle avoidance on nano-drones, but provide less spatial information than an 8×8 ToF array [5], [6].

B. Localisation and Mapping

Sensor streams are interpreted by visual-inertial odometry (VIO) and simultaneous localisation and mapping (SLAM) algorithms. VIO compares successive camera frames and fuses them with IMU data to estimate motion relative to the previous pose, as in the optimisation-based VINS-Mono [7] and Basalt, which also recovers non-linear factors from VIO for use in mapping [8]. Because

VIO integrates motion incrementally, it drifts over time. SLAM addresses this by using longer-term observations to maintain a globally consistent map, at the cost of greater computation and memory. ORB-SLAM3 is among the most accurate open-source visual and visual-inertial SLAM systems [9], and comparative evaluations show that performance varies considerably between sensor types and environments [1]. Hardware matters too: Delmerico and Scaramuzza found clear trade-offs between accuracy, efficiency and robustness for VIO on the embedded computers typical of flying robots [10]. Because the resulting pose is noisy and arrives at a lower rate than a flight controller needs, it is typically fused with IMU data in an Extended Kalman Filter (EKF) [11].

The map is commonly represented as an occupancy grid, which divides space into cells classified as occupied, free or unknown [12]. OctoMap extends this to 3D using an octree, keeping volumetric maps memory-efficient and explicitly modelling unknown space [13]. RTAB-Map builds a pose graph in which each node stores a keyframe and its estimated pose, and each edge stores the relative transformation between poses together with its uncertainty. It detects loop closures by appearance: when a new keyframe matches a previously visited location, a constraint is added and graph optimisation reduces the accumulated drift. Its memory management limits the active graph so that loop-closure detection remains real-time as the map grows, making it suitable for resource-constrained onboard computers [14]. SLAM's dependence on recognisable visual features, however, limits its performance in degraded inspection conditions such as dark or smoky areas, so relying solely on stereo-camera SLAM for collision prevention remains a safety concern.

C. Path Planning and Obstacle Avoidance

Global path planning over a SLAM map can only reason about known space, and in 3D the map is larger and may be wrong until enough data is gathered, so a planner must balance path quality, computation time and the ability to work on a map that is still growing. When the environment is unknown, the planner also needs somewhere to go: frontier-based exploration sends the robot to the boundaries between known free space and unmapped space, which are clustered into candidate targets in 3D [15] or maintained incrementally so targets update as the map grows [16]. The next target can be the nearest frontier, the one expected to reveal the most unmapped space, or a utility that trades this information gain against travel cost [17]. Next-best-view (NBV) planning also scores the sensor's orientation [18], which matters for a forward camera with a limited field of view [19].

Path planning algorithms generally fall into three categories [20]. Graph-based methods discretise the map into connected nodes. Dijkstra's algorithm expands outward from the start, always selecting the node with the lowest accumulated cost, which guarantees the lowest-cost path [21], and A* adds a heuristic estimate of the remaining distance that focuses the search while remaining optimal provided the heuristic never overestimates [22]. Costs can include both distance and proximity to obstacles, but the number of nodes and the memory required grow rapidly with map size in 3D. Metaheuristic approaches such as genetic algorithms evolve candidate paths scored against weighted criteria such as length, clearance and energy; they handle complex, non-linear constraints but do not guarantee the global optimum [20]. Sampling-based approaches such as Rapidly-exploring Random Trees (RRT) grow a tree from the UAV toward randomly sampled points, rejecting extensions that intersect obstacles [23]. They find paths quickly in high-dimensional, partially known spaces without a complete map, though not the optimal one. Modern quadrotor planners such as EGO-Planner rapidly re-optimise smooth, collision-free trajectories in unknown, cluttered environments using only the locally sensed map [24].

These advantages matter most in high-dimensional problems and agile flight. Sampling-based planners are known to struggle in narrow passages, where the probability of sampling inside the opening is small, and they return a different path on each replan, while EGO-Planner still relies on a graph search for its collision-free guide path [24]. For indoor inspection the search space is a position-only map a few metres across, and the critical cases are openings only slightly wider than the drone. This work therefore uses A*, which is deterministic and finds any opening that exists at the map resolution; Section V-B tests whether its cost is acceptable on a Raspberry Pi.

Local obstacle avoidance acts as a safety layer beneath the global planner, reacting continuously to sensor data so that the current motion does not lead into a sensed obstacle. Classic methods include artificial potential fields, which can become trapped in local minima [25], and the Vector Field Histogram, which selects a clear heading from a polar histogram of obstacle density [26]. Both layers are needed because the primary sensor may not update quickly enough or may fail to perceive certain geometry [6]. In this work the local layer is a clearance check that holds the drone when newly mapped obstacles invalidate its route, backed by the mechanical bump tolerance discussed next.

D. Bump Tolerance

Even with global and local avoidance, sensor uncertainty can lead to trajectories that cause collisions, and the local layer may not react quickly enough to prevent them, so a UAV in a cluttered environment must be able to absorb contact and continue. Design strategies should both reduce the likelihood of contact through avoidance and reduce its consequences through mechanical design. Commercial platforms show the latter: the industrial Flyability Elios 3 uses a carbon fibre cage for bump tolerance and crash resistance around about 2.4 kg of GPS-denied mapping hardware [27]. Research has explored several forms of collision resilience. Briod et al. developed the GimBall, a spherical cage decoupled from the vehicle by a gimbal that rolls along obstacles without destabilising flight, which directly preceded the Flyability platforms [28]. Mulgaonkar et al. showed that small caged quadrotors recover from collisions reliably enough that contact can be treated as part of normal operation [29].

Cages and propeller guards provide this safety buffer and offer mounting points for local sensing, but they add size, weight, cost and aerodynamic drag, trade-offs that matter in an environment that is already cluttered, potentially unknown and subject to unpredictable airflow. Brown et al. identified these limits when fully enveloping a drone within confined size limits [30]. Skarka et al. showed with ANSYS simulation that how connections and topology are defined determines how loads propagate through a drone cage [31], and Gobron et al. found through crash testing that damage can stay confined to a cage acting as a sacrificial barrier [32].

E. Research Gap

Collision-resilient designs such as the GimBall and Elios 3 rely on mechanical decoupling or expensive sensing [27], [28], low-cost confined-space platforms have added dedicated ranging sensors

to the camera [4], and stereo-only flight has relied on heavier computing hardware [3]. Agile planners such as EGO-Planner assume fast flight with a clean local map [24], and published cage analyses rarely compare numerical models against physical tests of friction-fit joints [31], [32]. This work therefore tests whether a single stereo camera and a Raspberry Pi 5 are enough for autonomous flight through openings only slightly wider than the drone, and evaluates a low-cost friction-fit cage both numerically and experimentally.

III. System Design

A. Electrical Design

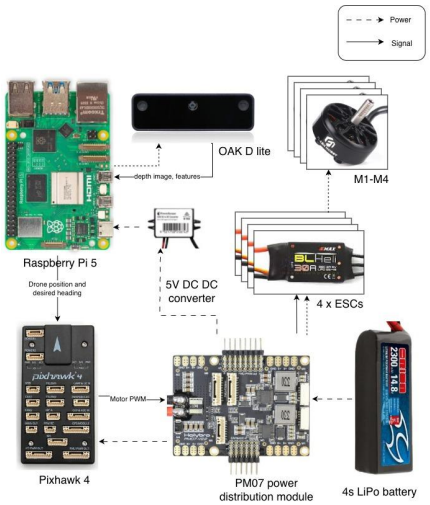


Fig. 1. Quadcopter electrical system architecture.

The electrical system architecture is outlined in Fig. 1. The OAK-D Lite stereo camera sends images, depth, tracked features and IMU data to the Raspberry Pi 5, which performs VIO and SLAM. The Pi outputs both a position measurement of the drone and position setpoints for the Pixhawk 4 flight controller. Motor PWM signals from the Pixhawk feed into the PM07 power distribution module and on to the ESCs, which drive the motors. The PM07 also powers the Pixhawk via a dedicated JST-GH port and powers the Pi through a 5 V, 5 A DC-DC converter attached to the PM07 battery terminals, which are fed by a 4S 5000 mAh LiPo battery. The Pi 5 was chosen over GPU-equipped computers such as the NVIDIA Jetson [33] for its lower cost and weight; Section V-D examines its headroom.

VL53L8CX ToF sensors [4] were initially planned for more precise obstacle detection, but flight testing showed, unexpectedly, that the SLAM map was sufficient to pass 0.53 m openings with the 0.5 m cage (Section V-B), so they were left out to save cost, complexity and weight.

B. Physical Design

The drone (Fig. 2) uses 6-inch propellers in a pusher configuration so that the arms attached to the carbon fibre cage are not shaken by the turbulent air below the propellers, driven by 1700 KV motors on a 4S battery. The frame holding the motors and electronics is made of PLA, which has a high stiffness but is more brittle than PETG. The stiffness is useful for reducing vibrations on the whole frame, but in retrospect PETG's toughness would have been more appropriate for a drone designed to be bump and crash tolerant. The battery is mounted at the bottom to keep the centre of mass low for flight stability, and the Pi and Pixhawk are stacked above it on 3D printed mounting plates. The all-up mass is 1.8 kg.

The cage, inspired by the Flyability Elios 3 [27], is made of 3 mm pultruded carbon fibre rods friction-fitted into PETG connectors. The rods are not permanently bonded, allowing limited movement under load. The SolidWorks assembly has 42 connectors and 120 rods; several connector variations are used, most notably the four modified connectors that attach the cage to the UAV's arms, as well as five- and six-socket arrangements. Connectors were 3D printed with a 0.2 mm layer height, 20% infill and 3 wall loops, and the sockets were post-drilled to 3 mm for a snug rod fit. Rods were cut to the centre-to-centre connector distances measured in SolidWorks.

A cage was chosen over propeller guards or ducts for two reasons. First, it surrounds the whole drone and so gives mounting points in every direction for added sensors, such as the ToF sensors considered for narrow openings (Section V-B). Second, ducts improve thrust efficiency only when the gap between the blade tips and the duct wall is very small [34], about 2 mm for these 6-inch propellers. A 3D-printed duct that tight would flex into the blades on impact, whereas the cage keeps contact well away from them. The cost is the cage's added mass and drag, which contribute to the endurance shortfall in Section V-D.

C. Software Architecture

The software is based on ROS 2 Jazzy [35] for its mature ecosystem and modular node-based architecture. The Pixhawk 4 runs the open-source PX4 flight firmware [11], which handles low-level flight dynamics and control, and communicates with the Pi over a uXRCE-DDS bridge that exposes PX4 messages as ROS 2 topics. Camera feeds and live maps are viewed on a ground computer through Foxglove.

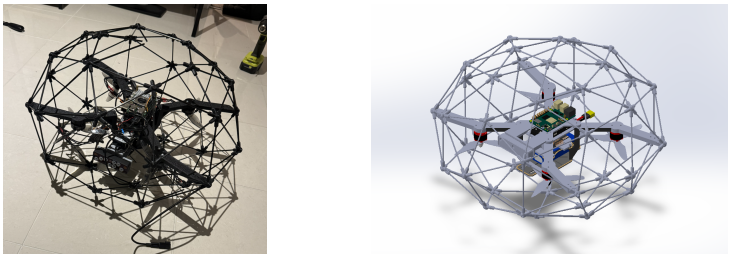


Fig. 2. The quadcopter (left) and its SolidWorks model (right).

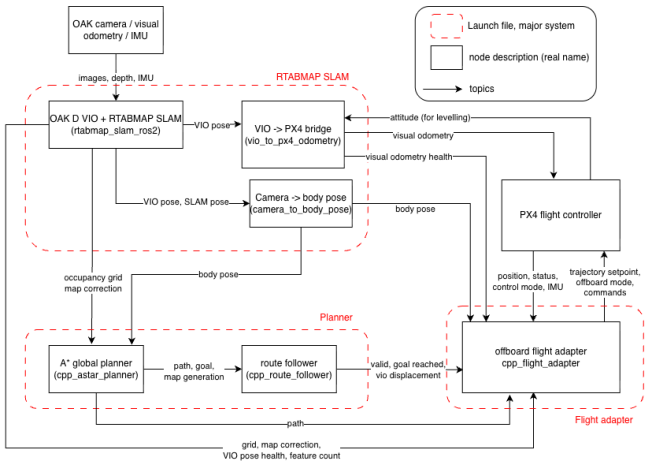


Fig. 3. Software architecture. The three main groups (SLAM, planner and flight adapter) are outlined in red.

The software is split into three main groups (Fig. 3). The *SLAM* group starts inside the camera: the OAK-D Lite contains an Intel Movidius Myriad X vision processor that detects Harris corners [36] and tracks them between frames with Lucas-Kanade optical flow [37] on board. RTAB-Map on the Pi estimates the camera's motion from these tracked features and the stereo depth, using a perspective-n-point solution with RANSAC outlier rejection, and fuses the camera IMU to make the estimate visual-inertial [14]. RTAB-Map also builds a persistent occupancy grid [12], in which each cell is classified as occupied, free or unknown, and maintains a pose graph whose loop closures correct accumulated drift.

Only the uncorrected odometry is sent to PX4, where the onboard EKF fuses it in place of GPS. Although it drifts over time, it is smooth and continuous, whereas loop-closure corrections cause sudden jumps in position that would destabilise the control loop. The bridge node converts the pose from ROS's ENU/FLU convention to PX4's NED/FRD, levels it with gravity using the attitude reported by PX4, and publishes a VIO health status.

The *planner* group runs A* [22] on RTAB-Map's occupancy grid from the drone's position to an operator-selected goal, replanning at 2 Hz. Obstacles are inflated by the drone's radius plus a safety margin, with an additional cost band that steers paths away from obstacles. The path is stored in the VIO frame and converted using RTAB-Map's map-to-odometry correction, so loop closures shift the path rather than invalidate it; if this correction stops, the planner stops publishing a path and the drone holds and lands. A route follower projects the drone onto the path at 10 Hz and places a rate-limited "carrot" target a fixed lookahead ahead. It outputs the *displacement* from the drone to the carrot rather than an absolute position.

The *flight adapter* is the only node that commands PX4. Its state machine streams offboard setpoints at 50 Hz, engages offboard mode, climbs to hover height and follows the route by adding the follower's displacement to PX4's own position estimate, so the controller never sees loop-closure jumps. It sends a position setpoint with velocity and acceleration feedforward and turns the drone to face along the path. It also monitors VIO feature count and freshness, tracking error, map correction size and route validity: if any check fails it holds position, and if the fault lasts longer than 3 s it hands control to PX4's automatic landing mode. Additional safeguards are a maximum flight time, a geofence, and manual land and kill commands.

IV. Experimental Methods

A. Localisation Against Motion Capture

The drone flew an autonomous 1 m square at a commanded height of 0.6 m, turning 90° on the spot at each corner for a full 360° of yaw. It was airborne for 56 s and flew 5.1 m at a mean speed of 0.09 m/s (maximum 0.28 m/s). The VIO pose was sent to PX4's EKF2 at 15 Hz, which fused it with the IMU for both horizontal position and height, and output its estimate at 50 Hz. A Vicon system tracked the airframe at 100 Hz as the reference. The Vicon and onboard logs share no clock, so they were synchronised by matching height profiles and then refined in 20 ms steps to minimise the 3D position error. Each onboard estimate was fitted to Vicon with a yaw rotation, a translation and a horizontal lever arm (the offset between the Vicon marker origin and the estimate's reference point); no scale factor was fitted. The VIO pose refers to the camera, about 0.10 m ahead of the flight controller, so it was given its own lever arm. Two error measures were computed over the airborne period, excluding 1 s at each end: *accuracy*, after aligning over the whole flight; and *drift*, after aligning on only the first 3 s, which approximates the error the drone builds up as it flies.

B. Navigation Flights

Navigation was tested with two planners that share the localisation stack and flight adapter. Two were used because most of an indoor route, like most drone flight, is flown at a roughly constant height, and only some sections, such as stairs or an obstacle to climb over, need a change in height. The 2D planner exploits this: it runs A* on RTAB-Map's occupancy grid (3 cm cells) at a fixed cruise height, a smaller, finer and cheaper search than a 3D map, with the carrot placed 0.25 m ahead of the drone and a cruise speed of 0.2 m/s. The 3D planner, for routes that must change height, runs A* through an OctoMap [13] built from the camera's depth images at a coarser 5 cm resolution, at a higher computational cost (Section V-D). A 3D route is accepted only if every point on it keeps at least 0.22 m between the drone's centre and the nearest occupied voxel, so the 0.25 m-radius cage needs an opening of about 0.5 m. In 3D the follower advances the setpoint along the route at a set speed (0.1–0.3 m/s) and slows it at corners, and the flight adapter holds position if a new route passes too close to newly mapped obstacles. Both planners treat unmapped space as untraversable; when a goal lies in unmapped space, the nearest reachable point, termed the *safe approach point*, is used instead. Path deviation is the horizontal distance from the drone to the path (2D) or to the route active 1 s earlier (3D), since each 3D replan starts the new route at the drone's current position. Clearances are read from the map and are therefore accurate to about one cell (±3 cm in 2D, ±2.5 cm in 3D).

C. Finite Element Analysis of the Cage

ANSYS Mechanical was used with a static structural analysis, which assumes no significant inertial effects. The whole cage was simplified to reduce detail among the abundant rod members: with this many connections, slipping and penetration were likely to build up, so 1D beam members of 3 mm circular section were used for the rods while the connectors kept their full solid PETG geometry, as they were the theorised weak point. This limits the model to bonded contacts, since without face features the beams cannot represent friction, so it is a stiffer idealisation of the cage. The four UAV-to-cage connectors were fixed and loads of 10–100 N were applied to a central, sideways-facing connector, probing equivalent von Mises stress and total and directional deformation.

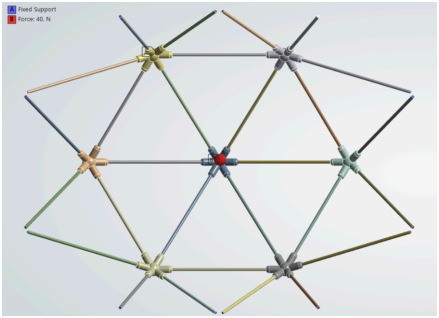


Fig. 4. Partial section model used for the detailed analysis and the static test, with fixed supports (A) at the loose rod ends and the load (B) on the central connector, here 40 N.

A second, partial model (Fig. 4) used the full detailed geometry with each loose rod end fixed, frictional contacts with an assumed

TABLE I
MATERIAL PROPERTIES USED IN ANSYS [38], [39]

Material	ρ (kg/m^3)	E (GPa)	ν	UTS (MPa)	UCS (MPa)	YS (MPa)
CF rod	1600	115	0.30	1700	450	–
PETG	1270	1.939	0.38	–	–	46.2

coefficient of 0.2, and loads on the central connector matching the test weights used experimentally. It was probed for the same quantities along with contact status, sliding, separation and penetration to verify the validity of the non-linear solution. Materials (Table I) were assumed linear elastic and isotropic. Large deflection was enabled so the stiffness matrix updated each iteration, and auto time stepping used 50 initial and minimum steps and up to 500 to ensure convergence. Mesh refinement on the whole-cage model at an excessive 2450 N load continued until stress and deformation changed by less than 5%, giving element sizes of 3.56 mm for the rods and 1.33 mm for the connectors.

D. Bump Test

Bump response was measured from the flight controller's own logs while the hovering drone (1.0 m on 3S; 0.3 and 0.8 m on 4S) was pushed sideways on the cage by hand. Because thrust acts only along the body z axis, the horizontal body-frame accelerometer reading a_{xy} measures the external force alone, so each push's velocity change and impulse are

$$\Delta v = \int_{\text{push}} a_{xy}\, dt, \qquad J = m\,\Delta v, \tag{1}$$

integrated over the samples where the horizontal force exceeded 5.4 N, with $m = 1.8$ kg (the 3S configuration may be slightly lighter, so Δv, which needs no mass, is used to compare the two). Displacement is PX4's position estimate minus the hold setpoint. The accelerometer is low-pass filtered at 30 Hz, which smooths short hand contacts, so impulse is the main measure of push size and peak forces are reported as filtered values. Pushes cut short by a landing or an EKF position reset were excluded from the statistics. The 3S tests used the earlier attitude and velocity gains (roll and pitch P of 4.0) and the 4S tests the retuned gains (6.5).

E. Static Deflection Test

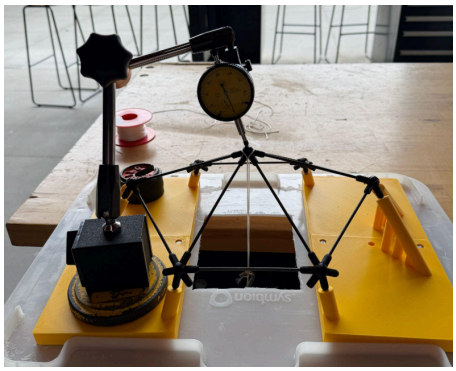


Fig. 5. Static deflection test setup.

To validate the partial model, a physical section with the same geometry was built (Fig. 5), fixed by rods extending from each connector into drilled PLA tubes in a four-piece stand bolted to a plastic lid. A cord tied around the central connector, laid around each extremity so the load was not applied to one side, passed through a hole in the lid to a hanger. A dial gauge with 0.01 mm resolution, zeroed before testing, contacted the central connector without touching the assembly. Weights from 100 g (the hanger) to 7 kg were applied in about 27 steps, letting the section settle before each reading, with $F = mg$ and $g = 9.81$ m/s^2.

F. Endurance and Computational Load

Endurance was measured in a 1.0 m autonomous hover on a fresh 4S pack until a loaded cell voltage of 3.50 V triggered a landing. CPU and memory use of each process were logged on the Pi during navigation flights.

G. Exploration in Simulation

The flights above fly to an operator-selected goal and stop at the safe approach point when it lies in unmapped space. Autonomous exploration was therefore developed in a 2D Python simulation, not yet on the drone. The simulated drone detects frontiers in its occupancy map, clusters them into candidate viewpoints and scores each by expected information gain, travel distance and obstacle clearance. A* plans a collision-free path to the best viewpoint through known space; the drone flies only a short segment of it, scans, updates its map and replans. This repeats until no reachable frontier remains. Coverage is measured as the fraction of the ground-truth free space that has been mapped.

V. RESULTS AND DISCUSSION

A. Localisation

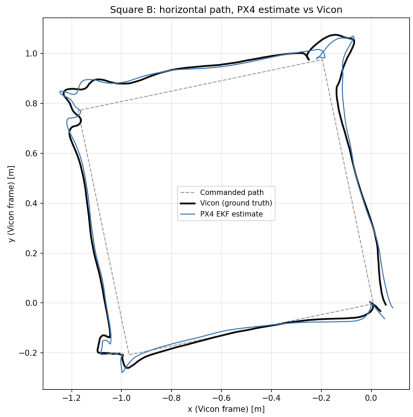


Fig. 6. Autonomous 1 m square: commanded path, Vicon ground truth and PX4 estimate.

TABLE II
PX4 POSITION ESTIMATE AGAINST VICON

Metric	**PX4 vs Vicon**
Horizontal error RMSE (max)	2.9 cm (6.9 cm)
Vertical error RMSE (max)	3.9 cm (9.5 cm)
Heading error RMSE (max)	0.42° (1.05°)
Horizontal drift	2.9 cm over 4.8 m (0.6%)
Height drift	15.6 cm/min fitted; 15 cm at end
Estimate rate (Vicon rate)	50 Hz (100 Hz)

The PX4 estimate stayed within 2.9 cm RMSE of Vicon horizontally, with a worst case of 6.9 cm (Table II, Fig. 6). A separate fit that allowed scale found the horizontal scale within 1.8% of unity. After aligning on the first 3 s only, the estimate ended

2.9 cm from the truth after 4.8 m of flight, or 0.6% of the distance. Heading agreed to within 0.42° RMSE over three 90° turns. The VIO input alone was almost as accurate (3.3 cm RMSE): with no GPS or magnetometer, the camera is the EKF's only absolute position reference, so the EKF raises the rate from 15 to 50 Hz and smooths the estimate but cannot remove error already in the VIO pose. The two other Vicon-tracked flights that day (a second square and a 3D-planner flight) gave 1.7 and 2.0 cm horizontal RMSE under an earlier, simpler alignment, so this flight is not a best case. Horizontally, aim 1 (within about 5 cm) is met over the short flights tested; over longer missions the 0.6% drift rather than the RMSE is the better guide, consistent with the accumulation of VIO error reported in benchmarks [10].

Path following is not localisation error. The flown square strayed from the commanded path by 5.0 cm typically (median), 11.9 cm at the 95th percentile and 14.9 cm at most, with overshoot at the corners. The PX4 estimate showed almost the same deviations (5.1 and 11.4 cm), correlated with the Vicon deviations at $r = 0.93$. PX4 therefore knew where it was, and these deviations come from the position controller rather than localisation.

Height is the weakest axis. EKF2 takes its height from the camera, so VIO height error passes straight through. PX4 believed it held 55–60 cm while Vicon showed the drone slowly rising from about 60 to 73 cm; the error fits a drift of 15.6 cm/min, reaching 15 cm by the end of the flight. The VIO input drifts the same way, which places the source in VIO rather than the EKF, and because the controller holds its estimated height, the real drone climbs as the estimate drifts down. The VIO frame was already levelled with PX4's attitude on this flight, and the error fits time far better than a tilted plane, so the cause is VIO height drift rather than tilt. For navigation it matters less than these numbers suggest, because the map drifts with the estimate (Section V-B), but it remains a limit wherever true height is needed, such as locating a defect on a roof, and for surfaces mapped long before. Fusing a second height reference, such as a downward rangefinder, is the obvious remedy.

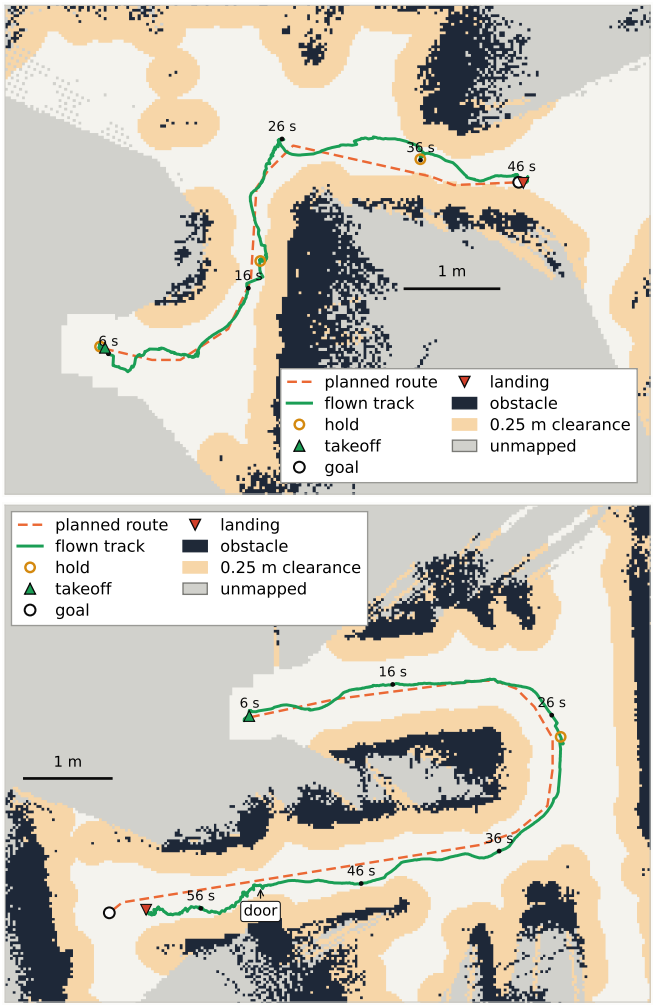


Fig. 7. Autonomous 2D flights, top view. (a, top) The room re-route flight re-routes as the map grows and reaches the goal. (b, bottom) The doorway flight passes along a corridor and through a doorway.

B. Navigation

TABLE III
AUTONOMOUS NAVIGATION FLIGHTS

Flight	Plan.	Dist. (m)	Time (s)	Outcome	Dev. (cm)	Holds (s)
Room re-route	2D	8.8	44	Goal, 9 cm	3 / 11	5.8
Doorway	2D	11.5	62	Landed 0.44 m short	2 / 5	0.7
Long corridor	2D	15.6	82	Stopped at doorway	2 / 6	6.8
Stairwell	3D	6.0†	46	Safe approach pt.	4 / 10	5.8
Box gap	3D	3.0	25	Goal	3 / 6	0.9
Obstacle climb	3D	6.5‡	37	Goal, 7 cm	3 / 6	2.8
Long 3D route	3D	7.4	81	Goal, 5 cm	2 / 6	15.1

Time runs from the first route command to arrival or landing. Dev. is the p50 / p95 horizontal distance to the path (2D) or to the route active 1 s earlier (3D). †+2.1 m climb. ‡+0.8 m climb. The doorway flight was landed with its route still valid; the long corridor flight stopped 3.4 m from its goal.

Seven autonomous flights tested the two planners in corridors, doorways, a stairwell and a gap between boxes (Table III). Four reached their goal, one reached its safe approach point as designed, one was landed with its route still valid at the end of the test, and one stopped at a doorway that had closed in the map.

Tracking and planning. The drone followed its routes to within 2–4 cm at the median and 5–11 cm at the 95th percentile. As in the Vicon square (Section V-A), these deviations come from the position control loop rather than from localisation. The setpoint moves continuously, so PX4 follows it with a small lag that grows on curves, overshoots at corners and stops, and is pushed sideways near walls and at the mouths of narrow openings, after which its velocity integrator over-corrects. Each replan also starts the new route from the drone's current position, so an offset at that moment carries into the new route. The flown tracks in Fig. 7 depart from the first planned routes by much more than this, up to about 30 cm, around 26–36 s in Fig. 7a and on the approach to the doorway in Fig. 7b. This is replanning, not tracking error: as the camera mapped obstacles the first plan could not see, the planner moved the route, and the drone stayed within 1–4 cm (median) of the route active at each moment. 2D replans took 2.5–7.6 ms at the median and under 17 ms at the 95th percentile, so the cost that reviews raise against graph search [20] is negligible at room scale; 3D A* is costlier (Section V-D) but still replans in flight.

Narrow openings. Tracking was tight enough to pass an opening with 1.5 cm to spare on each side, so whether an opening could be flown was decided by the map rather than by control (Table IV). In the box gap the planner found the opening without operator input and kept the drone's centre 0.25–0.29 m from the nearest occupied voxel (Fig. 8). This is narrower than the 0.6 m target of aim 2, achieved without the extra ranging sensors used in comparable low-cost platforms [4]. On the doorway flight the drone passed the doorway on its first plan, 9 cm off the centre line, without holding (Fig. 7b, Fig. 9).

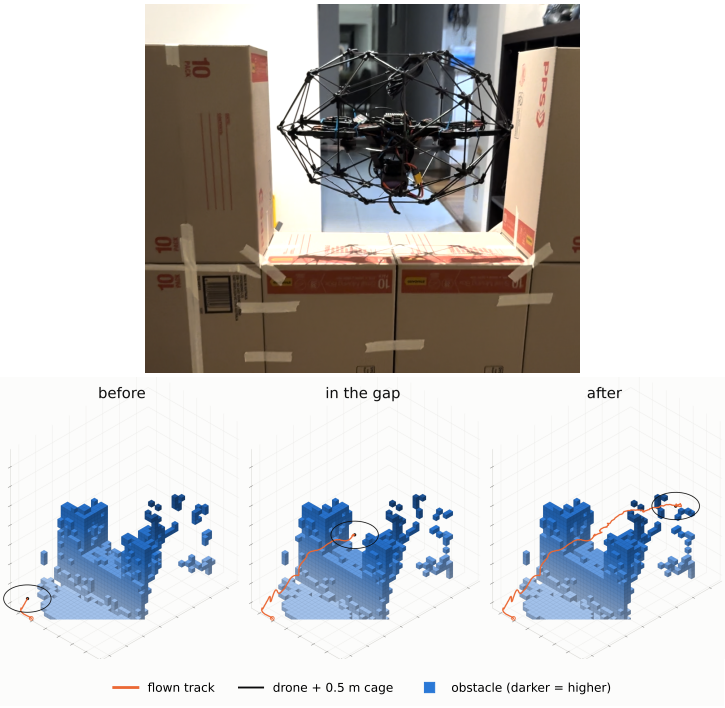


Fig. 8. Flight through a 0.53–0.60 m gap (box gap flight). (a, top) The gap between stacked boxes, with the 0.5 m cage for scale. (b, bottom) The OctoMap near the gap as mapped before, inside and after the gap, with the flown track and the cage outline. The drone stayed within about 3 cm of the gap's centre line.



Fig. 9. The doorway flight passing through the doorway. Past the door frame the camera faces a plain painted block wall.

TABLE IV
NARROW OPENINGS FLOWN

Opening (flight)	Width in map	Margin per side	Result
Gap between boxes (box gap)	0.53–0.60 m	1.5–5 cm	Passed, $\lesssim$3 cm off centre
Opening over obstacle (obstacle climb)	0.70 m	$\approx$10 cm	Passed
Doorway (doorway)	$\approx$0.9 m	$\approx$20 cm	Passed, no hold
Doorway (long corridor)	0.72–0.81 m	11–15 cm	Failed, closed in map

Margin per side is for the 0.5 m cage. Map widths are accurate to about one cell ($\pm$3 cm in 2D, $\pm$2.5 cm in 3D).

These margins are smaller than the localisation error measured against Vicon (2.9 cm RMSE, 6.9 cm worst case). This is possible because passing an opening depends on the drone's position relative to the opening, not on its absolute position. The map, the planned route and PX4's estimate all come from the same VIO frame, so an error common to them shifts the drone and the opening together and cancels. What remains is the drift built up between mapping the opening, about 1 m before entry, and flying through it, which at the measured 0.6% of distance is well under 1 cm, plus the tracking error and the accuracy of the map at the opening itself.

Failure: a doorway closed by stereo depth error. The one navigation failure came from the map, not the drone. On the long corridor flight, obstacle cells filled a 0.72–0.81 m doorway as the drone approached it, and the route through it disappeared (Fig. 10).

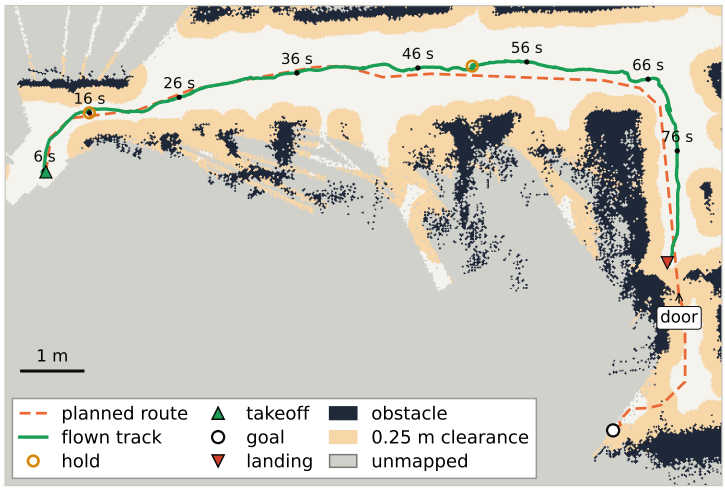


Fig. 10. Long corridor flight, top view, rotated so the corridor runs left to right. The first planned route passes through the doorway to the goal. By the end of the flight, map noise had closed the doorway, and the drone landed in front of it.

Once the drone was within about 1.5 m, obstacle cells built up in the opening: it narrowed to 0.69 m, then 0.60 m, and within a few seconds had closed completely. The planner could no longer find a route to the goal, its route shrank to nothing 0.5 m short of the doorway with the goal 3.4 m away, and the drone held and was promptly landed.

The doorway did not change, so the closure is attributed to stereo depth error on the plain wall beyond it. The doorway flight shows this kind of view (Fig. 9): past the frame the camera faces painted block work whose only texture is horizontal mortar lines, which run parallel to the stereo baseline and so give the matcher no disparity to measure. Stereo depth Z follows from the disparity d between the two images, and a disparity error δd gives a depth error δZ,

$$Z = \frac{fB}{d}, \qquad \delta Z \approx \frac{Z^2}{fB}\,\delta d, \tag{2}$$

where f is the focal length in pixels and B the stereo baseline, with $fB \approx 34.2$ px·m for the OAK-D Lite. From (2), a wall 3 m away has a disparity of about 11 px, and a mismatch of similar size places it at the doorway, 1.5 m away. Tracked features support this, falling from 320–460 per frame along the corridor to a mean of 238 at 1.5 m and 182 at 0.7 m (minimum 128) while the opening closed. A 0.33 m RTAB-Map correction as the drone approached the doorway may also have misaligned new surfaces.

This limits reliability rather than capability, since the drone passed the doorway on the doorway flight, where the map stayed clean. Because the planner treats every occupied cell as solid, a narrow opening can vanish exactly when the drone needs it, reflecting the known weakness of stereo on textureless surfaces [1], [2]. Depth filtering that rejects low-confidence matches, or a short-range ToF check of the opening before it is declared blocked [4], would address it.

Height changes. The 3D planner handled height changes without operator input. On the stairwell flight (Fig. 11) the drone climbed a route that rose 2.2 m over 2.9 m of horizontal distance, about 37°, reaching the safe approach point on the upper landing with its centre at least 0.29 m from the nearest mapped surface. Roughly 10–12 cm of height drift (Section V-A) had built up by the top, but it did not erode this clearance: the stairs, landing and ceiling were mapped in the same VIO frame as they came into view, so the drift moved them with the drone, as with the openings above.

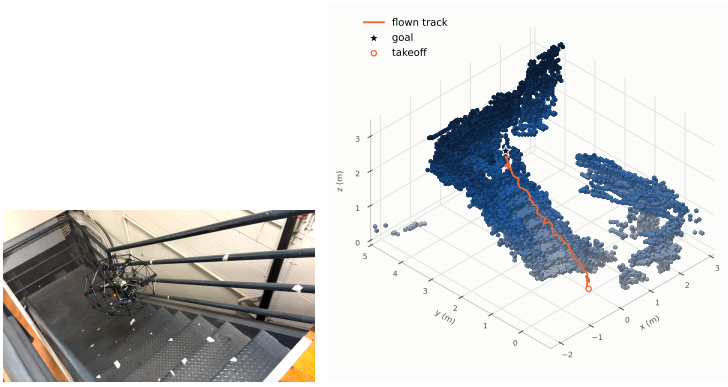


Fig. 11. Autonomous stair climb (stairwell flight). (a, left) The drone in the stairwell. (b, right) The OctoMap built during the flight (5 cm voxels, light is low and dark is high), with the flown track.

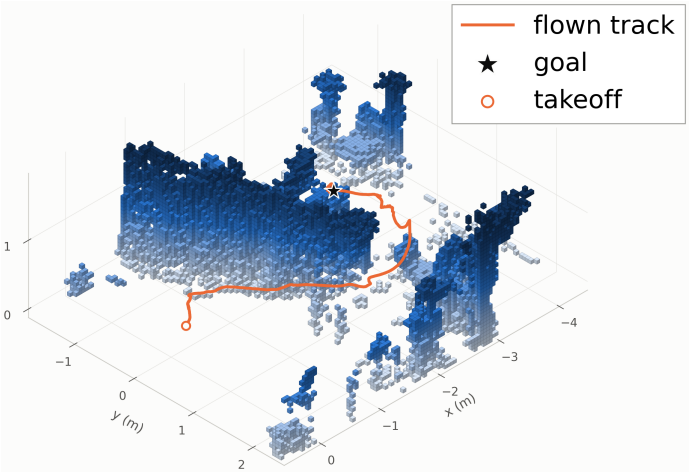


Fig. 12. Obstacle climb flight. The OctoMap built during the flight (5 cm voxels, light is low and dark is high), with the flown track from takeoff to goal.

Obstacle climb. The obstacle climb flight shows a longer 3D route through real clutter (Fig. 12). From takeoff the drone climbed 0.8 m, threaded over and between real obstacles including a net, passed through a 0.70 m opening (Table IV) and landed 7 cm from its goal, with the planner finding this route through the OctoMap without operator input.

C. Bump Tolerance

TABLE V
BUMP RESPONSE FROM FLIGHT-CONTROLLER LOGS

Metric	3S	4S
Pushes analysed	12	13
Velocity change Δv	0.29–2.48 m/s	0.29–1.94 m/s
Impulse J	0.5–4.5 N·s	0.5–3.5 N·s
Peak displacement	19–97 cm	7–62 cm
Displacement per Δv	0.36 s	0.27 s
Time to peak (median)	0.84 s	0.60 s
Back within 5 cm (median)	2.0 s	1.4 s
Overshoot, median (max)	11 (44) cm	6 (18) cm
Peak force, median (max)	71 (194) N	50 (128) N

Peak force after the 30 Hz filter; true peaks are likely higher.

Bump response. Table V summarises the 25 pushes on the cage that were analysed. The drone stayed under control in all 33 recorded pushes, consistent with treating contact as part of normal operation [29], and peak displacement scaled almost linearly with push size. On 4S it moved a quarter less per unit Δv, returned within 5 cm faster and overshot about half as far, though the gains were retuned between the two configurations, so the battery and tuning share the credit. Four pushes triggered an automatic landing from the companion software's safety checks, including its 60°/s yaw-rate limit, as off-centre pushes spin the drone.

TABLE VI
WHOLE-CAGE ANSYS PREDICTIONS

Load (N)	Max. stress (MPa)	Y deform. (mm)
11.8	0.250	0.014
23.7	0.501	0.028
79.8	1.687	0.094

Cage structure. The whole-cage model predicted little structural response across its 10–100 N sweep (Table VI): 0.501 MPa and 0.028 mm at 23.7 N, rising to 1.687 MPa and 0.094 mm at 79.8 N, almost linearly with load. This is far below the 46.2 MPa PETG yield strength. The pushes reached filtered peak forces of up to about 200 N (Table V), beyond the modelled range, but even there the linear trend gives only about 4 MPa. Yet on the highest-force push a connector failed (Fig. 13). The failure shows why: the printed socket arms sheared off where they join the hub, leaving their ends on the rods. This is local loading within individual sockets, across the print layers, which neither bonded beam members nor an isotropic material model can represent.



Fig. 13. Connector failure on the highest-force push. The printed socket arms sheared off at the hub, leaving their ends on the rods.

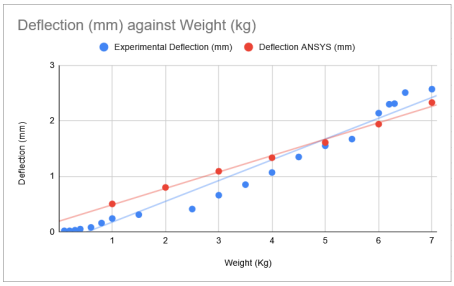


Fig. 14. Measured versus ANSYS deflection of the central connector in the static test.

The detailed partial model, in which frictional contacts allowed movement between rods and connectors, was compared with the static test (Fig. 14). At low loads ANSYS overpredicted the deflection, giving 0.505 mm at 1 kg against 0.24 mm measured and 1.334 mm at 3 kg against 0.56–0.66 mm. This suggests the assumed friction coefficient of 0.2 allowed more local movement than the snug physical fit, again showing the sensitivity of the cage response to how its connections are represented [31]. The two curves cross at about 5 kg, beyond which the physical section deformed non-linearly and exceeded the prediction, reaching 2.57 mm at 7 kg.

After unloading, it returned to within 0.03 mm of its original position, which was attributed to support settling, and no permanent damage was observed.

Static strength alone therefore does not describe the cage's survivability, and experimental testing remains essential for a structure meant to tolerate collision. For aim 3, the cage kept the drone within its 0.5 m envelope and let it recover from every push without losing control; the single connector failure is consistent with the sacrificial role described in [32], and thicker socket roots or solid infill around the sockets would raise the force at which they fail.

D. Cost, Endurance and Computational Load

TABLE VII
BILL OF MATERIALS

Subsystem	Part	AUD
Frame	F330 quadcopter frame	7.75
Flight controller	Holybro Pixhawk 4	211
Power module	Holybro PM07	78
Companion computer	Raspberry Pi 5	250
Camera	Luxonis OAK-D Lite	266
Motors	4 × brushless, 1700 KV	65
ESCs	4 × BLHeli 30 A	86
Propellers	4 × 6-inch	5
Battery	Turnigy Graphene Panther 4S 5000 mAh 100C	88
Pi power	5 V BEC from the flight battery	28
Cage	3 mm pultruded carbon fibre rods	30
Total	All-up mass 1.8 kg	**1114.75**

Cost. The parts cost AUD 1114.75 (Table VII), 11% over the AUD 1000 target of aim 4. The OAK-D Lite and Raspberry Pi 5 alone account for almost half of this, so the target is missed by the perception and computing hardware rather than by the airframe. Both prices were also raised by the rising cost of memory during the project, since the Pi 5 and the OAK-D Lite each carry their own RAM.

Endurance. The hover test lasted 6 min 56 s before the 3.50 V per cell landing threshold, drawing 4321 mAh from the 5000 mAh pack at a mean of 37.4 A (546 W). Endurance therefore falls short of the 10 minute target in aim 4; the cage, 4S propulsion and 1.8 kg all-up mass are the main contributors, and a larger pack or lighter frame would be needed to close the gap.

Computational load. During a 2D flight the Pi used 77% of its four cores on average and 81% at peak (Table VIIIa). SLAM and VIO took over half of the machine, while the C++ planner, follower and flight adapter together used under 4%. The 3D stack (Table VIIIb) averaged slightly less, 72%, but peaked at 97%. The 3D stack does strictly more work: the same VIO, the same SLAM, and on top of them OctoMap (42%) and 3D A* (41%). Its average is similar because of how the SLAM is run. In the 2D stack the RTAB-Map SLAM node inside the DepthAI pipeline keeps one core saturated, although it adds map keyframes at only 1 Hz. For the 3D stack the ROS RTAB-Map mapper was configured to match it (the same keyframe rate, robot footprint and map outputs), which allowed the DepthAI SLAM node to be switched off. The ROS mapper does the same mapping for 28% of a core, so camera, VIO and SLAM together fall from 210% to 127%, and the core this frees pays for the 3D map and planner. The two stacks differ more at peak: OctoMap and 3D A* burst to 98% and 77% of a core as the map updated (0.9 Hz) and the route was replanned (1.4 Hz). Memory stayed below 40% of 7.9 GB in both, and the processor never throttled. The Pi 5 is therefore sufficient but near its limit in 3D, and a GPU-equipped computer [33] would be the upgrade path if more sensing were added.

TABLE VIII
RASPBERRY PI 5 LOAD DURING 2D AND 3D NAVIGATION FLIGHTS

Process	CPU, % of one core (avg / max)	Memory (MB)
(a) 2D stack		
SLAM + VIO (RTAB-Map)	210 / 219	412
Bag recorder	28 / 31	88
VIO → PX4 bridge	16 / 18	84
PX4 pose relay	9 / 10	83
Flight adapter (C++)	8 / 9	40
XRCE agent	7 / 9	27
A* planner (C++)	3 / 4	36
Route follower (C++)	3 / 4	35
Foxglove bridge	1 / 2	48
Battery monitor	0.4 / 1	80
Whole Pi, % of 4 cores	77 / 81	3132
(b) 3D stack, obstacle climb flight		
Camera + RTAB-Map VIO	99 / 122	349
OctoMap	42 / 98	307
3D A* planner	41 / 77	104
RTAB-Map mapper	28 / 70	352
Bag recorder	19 / 24	131
Flight adapter	16 / 33	90
VIO → PX4 bridge	3 / 7	85
Foxglove bridge	2 / 3	50
Planner pose relay	1 / 2	33
Whole Pi, % of 4 cores	72 / 97	2801

The same load explains the holds (0.7–15 s per flight, Table III). When newly mapped voxels make the next route segment fail the clearance check, the flight adapter stops the drone until a valid route arrives, and with the map updating at 0.9 Hz and routes at 1.4 Hz each hold lasts at least one update, giving a stop-start motion. Faster mapping and replanning, which the CPU headroom limits, would shorten them.

E. Autonomous Exploration in Simulation

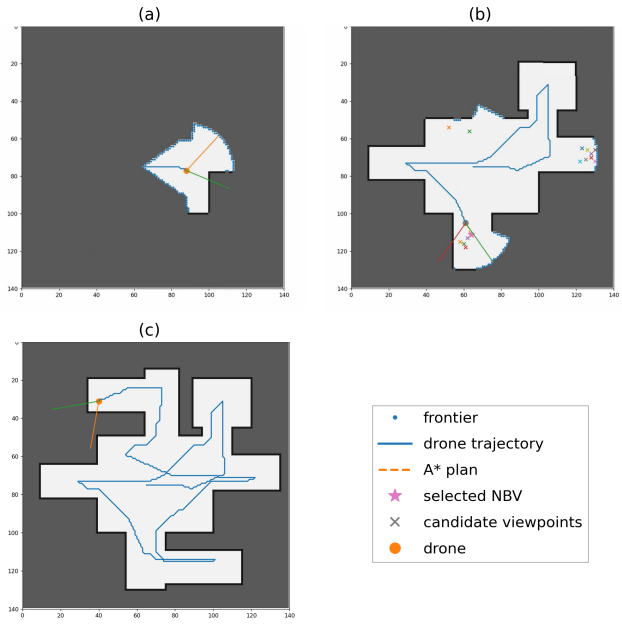


Fig. 15. Frontier exploration with NBV selection and A* in the 2D simulation: frontiers, flown trajectory and drone position after (a) 7 frames, 15.5% of the free space mapped; (b) 142 frames, 77.6%, with the candidate viewpoints (crosses), the selected NBV and its A* plan; and (c) 325 frames, 100%.

Starting from its first scan, which covered 15.5% of the free space, the simulated drone mapped 77.6% by frame 142 and all of the accessible free space by frame 325 (Fig. 15). The trajectory

crosses explored areas several times between distant frontiers, so target selection could be more efficient. The simulation used an idealised map and sensor, and exploration time, distance flown and computation were not yet measured; integrating it with the ROS 2 and RTAB-Map stack and testing on the drone are the next steps.

F. Limitations and Future Work

The flights in Table III were selected to show the range of behaviour, including a failure, so a success rate over repeated trials remains to be measured, and ground truth covered short flights only. The forward-facing camera knows obstacles beside, above or behind the drone only if it saw them earlier. Future work should add a second height reference for longer missions and true-height measurements, depth filtering or ToF checks at narrow openings, and a longer-endurance power system. An EGO-Planner-style optimiser [24] could also smooth the stop-start motion through holds.

VI. Conclusion

This paper presented a quadcopter costing about AUD 1115 that uses a single OAK-D Lite stereo camera and a Raspberry Pi 5 to localise, map and navigate cluttered GPS-denied indoor spaces, enclosed in a friction-fit carbon fibre cage. Against the four aims: localisation stayed within 2.9 cm RMSE of Vicon horizontally with 0.6% drift, meeting the 5 cm target, although height drifted by about 15 cm/min; the drone navigated autonomously through a 0.53–0.60 m gap with 1.5–5 cm per side, through a doorway and up a 2.1 m stair climb, but one doorway was lost when stereo errors on a textureless wall closed it in the map; the drone survived bumps on its cage of up to about 200 N; and the platform came in 11% over its AUD 1000 cost target and reached about 7 rather than 10 minutes of hover. Stereo-only autonomy on low-cost hardware is therefore feasible in the environments tested, without the ranging sensors originally planned, and map reliability at narrow openings, height drift and endurance are the remaining limits.

Acknowledgment

Generative AI (Anthropic's Claude, used through Claude Code) assisted with software development, flight-log analysis, figure generation and editing parts of this paper. The authors checked all results against the flight logs and take full responsibility for the content.

References

[1] B. Garigipati, N. Strokina, and R. Ghabcheloo, "Evaluation and comparison of eight popular Lidar and visual SLAM algorithms," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Kyoto, Japan, Oct. 2022.

[2] M. Z. Butt, N. Nasir, and R. B. A. Rashid, "A review of perception sensors, techniques, and hardware architectures for autonomous low-altitude UAVs in non-cooperative local obstacle avoidance," *Robot. Auton. Syst.*, vol. 173, Art. no. 104629, Mar. 2024.

[3] K. Schmid, T. Tomic, F. Ruess, H. Hirschmüller, and M. Suppa, "Stereo vision based indoor/outdoor navigation for flying robots," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2013, pp. 3955–3962.

[4] S. Basnet, J. C. Andersen, and E. Boukas, "Design and implementation of a low-cost perception system for aerial robots in confined spaces," *Sensors*, vol. 26, no. 4, Art. no. 1140, Feb. 2026.

[5] H. Müller, V. Kartsch, M. Magno, and L. Benini, "BatDeck: Ultra low-power ultrasonic ego-velocity estimation and obstacle avoidance on nano-drones," arXiv:2412.10048, Dec. 2024.

[6] A. Merei, H. Mcheick, A. Ghaddar, and D. Rebaine, "A survey on obstacle detection and avoidance methods for UAVs," *Drones*, vol. 9, no. 3, Art. no. 203, 2025.

[7] T. Qin, P. Li, and S. Shen, "VINS-Mono: A robust and versatile monocular visual-inertial state estimator," *IEEE Trans. Robot.*, vol. 34, no. 4, pp. 1004–1020, Aug. 2018.

[8] V. Usenko, N. Demmel, D. Schubert, J. Stückler, and D. Cremers, "Visual-inertial mapping with non-linear factor recovery," *IEEE Robot. Autom. Lett.*, vol. 5, no. 2, pp. 422–429, Apr. 2020.

[9] C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. M. Montiel, and J. D. Tardós, "ORB-SLAM3: An accurate open-source library for visual, visual-inertial, and multimap SLAM," *IEEE Trans. Robot.*, vol. 37, no. 6, pp. 1874–1890, Dec. 2021.

[10] J. Delmerico and D. Scaramuzza, "A benchmark comparison of monocular visual-inertial odometry algorithms for flying robots," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2018, pp. 2502–2509.

[11] L. Meier, D. Honegger, and M. Pollefeys, "PX4: A node-based multithreaded open source robotics framework for deeply embedded platforms," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2015, pp. 6235–6240.

[12] A. Elfes, "Using occupancy grids for mobile robot perception and navigation," *Computer*, vol. 22, no. 6, pp. 46–57, Jun. 1989.

[13] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss, and W. Burgard, "OctoMap: An efficient probabilistic 3D mapping framework based on octrees," *Auton. Robots*, vol. 34, no. 3, pp. 189–206, 2013.

[14] M. Labbé and F. Michaud, "RTAB-Map as an open-source lidar and visual simultaneous localization and mapping library for large-scale and long-term online operation," *J. Field Robot.*, vol. 36, no. 2, pp. 416–446, 2019.

[15] A. Batinovic, T. Petrovic, A. Ivanovic, F. Petric, and S. Bogdan, "A multi-resolution frontier-based planner for autonomous 3D exploration," *IEEE Robot. Autom. Lett.*, vol. 6, no. 3, pp. 4528–4535, Jul. 2021.

[16] B. Zhou, Y. Zhang, X. Chen, and S. Shen, "FUEL: Fast UAV exploration using incremental frontier structure and hierarchical planning," *IEEE Robot. Autom. Lett.*, vol. 6, no. 2, pp. 779–786, Apr. 2021.

[17] A. Dai, S. Papatheodorou, N. Funk, D. Tzoumanikas, and S. Leutenegger, "Fast frontier-based information-driven autonomous exploration with an MAV," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2020, pp. 9570–9576.

[18] A. Batinovic, A. Ivanovic, T. Petrovic, and S. Bogdan, "A shadowcasting-based next-best-view planner for autonomous 3D exploration," *IEEE Robot. Autom. Lett.*, vol. 7, no. 2, pp. 2969–2976, Apr. 2022.

[19] Y. Zhao, L. Yan, Y. Chen, H. Xie, and B. Xu, "FAEP: Fast autonomous exploration planner for UAV equipped with limited FOV sensor," arXiv:2202.12507, 2022.

[20] M. Nafees, T. Syeda, A. Ali, H. Farman, and M. Tayyab, "An in-depth analysis of UAV path planning, including procedures, algorithms, optimization models, and emerging challenges," *MethodsX*, vol. 16, Art. no. 103826, Jun. 2026.

[21] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numer. Math.*, vol. 1, pp. 269–271, 1959.

[22] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, pp. 100–107, 1968.

[23] S. M. LaValle, "Rapidly-exploring random trees: A new tool for path planning," Iowa State Univ., Ames, IA, USA, Tech. Rep. TR 98-11, 1998.

[24] X. Zhou, Z. Wang, H. Ye, C. Xu, and F. Gao, "EGO-Planner: An ESDF-free gradient-based local planner for quadrotors," *IEEE Robot. Autom. Lett.*, vol. 6, no. 2, pp. 478–485, Apr. 2021.

[25] O. Khatib, "Real-time obstacle avoidance for manipulators and mobile robots," *Int. J. Robot. Res.*, vol. 5, no. 1, pp. 90–98, 1986.

[26] J. Borenstein and Y. Koren, "The vector field histogram: Fast obstacle avoidance for mobile robots," *IEEE Trans. Robot. Autom.*, vol. 7, no. 3, pp. 278–288, Jun. 1991.

[27] Flyability, "Flyability launches the Elios 3, an indoor LiDAR drone for Industry 4.0," May 19, 2022. [Online]. Available: https://www.flyability.com/news/elios-3-launch

[28] A. Briod, P. Kornatowski, J.-C. Zufferey, and D. Floreano, "A collision-resilient flying robot," *J. Field Robot.*, vol. 31, no. 4, pp. 496–509, 2014.

[29] Y. Mulgaonkar, A. Makineni, L. Guerrero-Bonilla, and V. Kumar, "Robust aerial robot swarms without collision avoidance," *IEEE Robot. Autom. Lett.*, vol. 3, no. 1, pp. 596–603, Jan. 2018.

[30] L. Brown *et al.*, "The design of Prometheus: A reconfigurable UAV for subterranean mine inspection," *Robotics*, vol. 9, no. 4, Art. no. 95, 2020.

[31] W. Skarka *et al.*, "Load analysis on the drone protection cage increasing collision resistance," *Procedia Struct. Integrity*, vol. 54, pp. 506–513, 2024.

[32] S. Gobron, I. Serra Moncadas, T. Flück, J. Voëffray, and M. Lauria, "A graph-based workflow for the design and prototyping of protective cages for UAVs," *Drones*, vol. 10, no. 10, Art. no. 734, 2026.

[33] NVIDIA, "Jetson for next-generation robotics." [Online]. Available: https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/

[34] J. L. Pereira and I. Chopra, "Hover tests of micro aerial vehicle-scale shrouded rotors, Part I: Performance characteristics," *J. Amer. Helicopter Soc.*, vol. 54, no. 1, Art. no. 012001, 2009.

[35] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," *Sci. Robot.*, vol. 7, no. 66, 2022.

[36] C. Harris and M. Stephens, "A combined corner and edge detector," in *Proc. Alvey Vis. Conf.*, 1988, pp. 23.1–23.6.

[37] B. D. Lucas and T. Kanade, "An iterative image registration technique with an application to stereo vision," in *Proc. 7th Int. Joint Conf. Artif. Intell. (IJCAI)*, Vancouver, BC, Canada, 1981, pp. 674–679.

[38] Fibrolux GmbH, "Pultruded carbon tubes/rods." [Online]. Accessed: Oct. 11, 2026.

[39] UltiMaker, "UltiMaker PETG technical data sheet," ver. 1.00, Apr. 2022.